In [1]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import confusion_matrix, accuracy_score
```

In [2]:
```
pip install tensorflow
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: tensorflow in c:\users\hp\appdata\roaming\python\pyth
on313\site-packages (2.21.0)
Requirement already satisfied: absl-py>=1.0.0 in c:\users\hp\appdata\roaming\python
\python313\site-packages (from tensorflow) (2.5.1)
Requirement already satisfied: astunparse>=1.6.0 in c:\users\hp\appdata\roaming\pyth
on\python313\site-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=25.9.23 in c:\users\hp\appdata\roaming\p
ython\python313\site-packages (from tensorflow) (25.12.19)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in c:\users\hp\ap
pdata\roaming\python\python313\site-packages (from tensorflow) (0.7.0)
Requirement already satisfied: google_pasta>=0.1.1 in c:\users\hp\appdata\roaming\py
thon\python313\site-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in c:\users\hp\appdata\roaming\pytho
n\python313\site-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt_einsum>=2.3.2 in c:\users\hp\appdata\roaming\pyth
on\python313\site-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in c:\users\hp\appdata\roaming\python\pytho
n313\site-packages (from tensorflow) (25.0)
Requirement already satisfied: protobuf<8.0.0,>=6.31.1 in c:\users\hp\appdata\roamin
g\python\python313\site-packages (from tensorflow) (7.36.2)
Requirement already satisfied: requests<3,>=2.21.0 in c:\users\hp\appdata\roaming\py
thon\python313\site-packages (from tensorflow) (2.32.5)
Requirement already satisfied: setuptools in c:\users\hp\appdata\roaming\python\pyth
on313\site-packages (from tensorflow) (80.9.0)
Requirement already satisfied: six>=1.12.0 in c:\users\hp\appdata\roaming\python\pyt
hon313\site-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in c:\users\hp\appdata\roaming\pytho
n\python313\site-packages (from tensorflow) (3.3.0)
Requirement already satisfied: typing_extensions>=3.6.6 in c:\users\hp\appdata\roami
ng\python\python313\site-packages (from tensorflow) (4.15.0)
Requirement already satisfied: wrapt>=1.11.0 in c:\programdata\anaconda3\lib\site-pa
ckages (from tensorflow) (1.17.0)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in c:\users\hp\appdata\roaming\py
thon\python313\site-packages (from tensorflow) (1.84.0)
Requirement already satisfied: keras>=3.12.0 in c:\users\hp\appdata\roaming\python\p
ython313\site-packages (from tensorflow) (3.15.1)
Requirement already satisfied: numpy>=1.26.0 in c:\users\hp\appdata\roaming\python\p
ython313\site-packages (from tensorflow) (2.3.0)
Requirement already satisfied: h5py<3.15.0,>=3.11.0 in c:\users\hp\appdata\roaming\p
ython\python313\site-packages (from tensorflow) (3.14.0)
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in c:\users\hp\appdata\roamin
g\python\python313\site-packages (from tensorflow) (0.6.0)
Requirement already satisfied: charset_normalizer<4,>=2 in c:\users\hp\appdata\roami
ng\python\python313\site-packages (from requests<3,>=2.21.0->tensorflow) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in c:\users\hp\appdata\roaming\python\py
thon313\site-packages (from requests<3,>=2.21.0->tensorflow) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\hp\appdata\roaming\pyt
hon\python313\site-packages (from requests<3,>=2.21.0->tensorflow) (2.6.2)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\hp\appdata\roaming\pyt
hon\python313\site-packages (from requests<3,>=2.21.0->tensorflow) (2025.11.12)
Requirement already satisfied: wheel<1.0,>=0.23.0 in c:\programdata\anaconda3\lib\si
te-packages (from astunparse>=1.6.0->tensorflow) (0.45.1)
Requirement already satisfied: rich in c:\programdata\anaconda3\lib\site-packages (f
rom keras>=3.12.0->tensorflow) (14.2.0)
Requirement already satisfied: namex in c:\users\hp\appdata\roaming\python\python313
```

```
\site-packages (from keras>=3.12.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in c:\users\hp\appdata\roaming\python\python31
3\site-packages (from keras>=3.12.0->tensorflow) (0.20.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in c:\programdata\anaconda3\lib
\site-packages (from rich->keras>=3.12.0->tensorflow) (2.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in c:\users\hp\appdata\roamin
g\python\python313\site-packages (from rich->keras>=3.12.0->tensorflow) (2.19.2)
Requirement already satisfied: mdurl~=0.1 in c:\programdata\anaconda3\lib\site-packa
ges (from markdown-it-py>=2.2.0->rich->keras>=3.12.0->tensorflow) (0.1.2)
Note: you may need to restart the kernel to use updated packages.
[notice] A new release of pip is available: 25.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

In [4]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
```

In [5]:
```
df = pd.read_csv("Churn_Modelling.csv")
```

In [6]:
```
df.head()
```

Out[6]:

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	B
0	1	15634602	Hargrave	619	France	Female	42	2	
1	2	15647311	Hill	608	Spain	Female	41	1	83
2	3	15619304	Onio	502	France	Female	42	8	159
3	4	15701354	Boni	699	France	Female	39	1	
4	5	15737888	Mitchell	850	Spain	Female	43	2	125

In [7]:
```
# 2. Distinguish the feature and target set
X = df.iloc[:, 3:13]   # Features (from CreditScore to EstimatedSalary)
y = df.iloc[:, 13]     # Target (Exited column)
```

In [8]:
```
# Encoding categorical variables (Geography and Gender)
labelencoder_gender = LabelEncoder()
X['Gender'] = labelencoder_gender.fit_transform(X['Gender'])
```

In [9]:
```
# One-hot encode Geography
X = pd.get_dummies(X, columns=['Geography'], drop_first=True)
```

In [10]:
```
# 3. Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
```

In [11]:
```
# Normalize the data
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

In [12]:
```
X_train_df = pd.DataFrame(X_train, columns=X.columns)
```

```
print("Sample of normalized training data:")
display(X_train_df.head())

print("\nMean of features after scaling:\n", X_train_df.mean())
print("\nStandard deviation of features after scaling:\n", X_train_df.std())
```

Sample of normalized training data:

	CreditScore	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsAct
0	0.356500	0.913248	-0.655786	0.345680	-1.218471	0.808436	0.649203	
1	-0.203898	0.913248	0.294938	-0.348369	0.696838	0.808436	0.649203	
2	-0.961472	0.913248	-1.416365	-0.695393	0.618629	-0.916688	0.649203	
3	-0.940717	-1.094993	-1.131148	1.386753	0.953212	-0.916688	0.649203	
4	-1.397337	0.913248	1.625953	1.386753	1.057449	-0.916688	-1.540351	

```
Mean of features after scaling:
 CreditScore          5.435652e-16
Gender                2.842171e-17
Age                  -1.896261e-16
Tenure                6.661338e-17
Balance              -3.099743e-16
NumOfProducts         1.065814e-16
HasCrCard            -3.197442e-17
IsActiveMember       -1.776357e-17
EstimatedSalary       1.776357e-17
Geography_Germany    -4.263256e-17
Geography_Spain      -7.194245e-17
dtype: float64

Standard deviation of features after scaling:
 CreditScore           1.000063
Gender                 1.000063
Age                    1.000063
Tenure                 1.000063
Balance                1.000063
NumOfProducts          1.000063
HasCrCard              1.000063
IsActiveMember         1.000063
EstimatedSalary        1.000063
Geography_Germany      1.000063
Geography_Spain        1.000063
dtype: float64
```

In [13]:
```
# 4. Initialize and build the Neural Network model
model = Sequential()
model.add(Dense(units=6, activation='relu', input_dim=X_train.shape[1]))
model.add(Dropout(0.3))
model.add(Dense(units=6, activation='relu'))
model.add(Dense(units=1, activation='sigmoid'))
```

```
C:\Users\HP\AppData\Roaming\Python\Python313\site-packages\keras\src\layers\core\dense.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

In [15]:
```
# Compile the model
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

print("Model compiled successfully!")
```

```
Model compiled successfully!
```

In [16]:
```
# Train for 20 epochs
history = model.fit(X_train, y_train, epochs=20, batch_size=32, validation_data=(X_
```

```
Epoch 1/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step - accuracy: 0.7470 - loss: 0.5770 - val_acc
uracy: 0.8035 - val_loss: 0.5094
Epoch 2/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.7945 - loss: 0.5044 - val_acc
uracy: 0.8035 - val_loss: 0.4728
Epoch 3/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.7955 - loss: 0.4821 - val_acc
uracy: 0.8035 - val_loss: 0.4478
Epoch 4/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.7977 - loss: 0.4634 - val_acc
uracy: 0.8150 - val_loss: 0.4339
Epoch 5/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8048 - loss: 0.4545 - val_acc
uracy: 0.8185 - val_loss: 0.4235
Epoch 6/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8060 - loss: 0.4490 - val_acc
uracy: 0.8185 - val_loss: 0.4210
Epoch 7/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8099 - loss: 0.4447 - val_acc
uracy: 0.8215 - val_loss: 0.4162
Epoch 8/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8106 - loss: 0.4374 - val_acc
uracy: 0.8220 - val_loss: 0.4125
Epoch 9/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8117 - loss: 0.4379 - val_acc
uracy: 0.8265 - val_loss: 0.4083
Epoch 10/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8119 - loss: 0.4378 - val_acc
uracy: 0.8295 - val_loss: 0.4073
Epoch 11/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8160 - loss: 0.4308 - val_acc
uracy: 0.8290 - val_loss: 0.4034
Epoch 12/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8167 - loss: 0.4289 - val_acc
uracy: 0.8315 - val_loss: 0.4025
Epoch 13/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8207 - loss: 0.4291 - val_acc
uracy: 0.8315 - val_loss: 0.3998
Epoch 14/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8170 - loss: 0.4279 - val_acc
uracy: 0.8350 - val_loss: 0.3972
Epoch 15/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8209 - loss: 0.4263 - val_acc
uracy: 0.8370 - val_loss: 0.3951
Epoch 16/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8210 - loss: 0.4238 - val_acc
uracy: 0.8380 - val_loss: 0.3956
Epoch 17/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8201 - loss: 0.4227 - val_acc
uracy: 0.8405 - val_loss: 0.3930
Epoch 18/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8213 - loss: 0.4205 - val_acc
uracy: 0.8420 - val_loss: 0.3886
Epoch 19/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8267 - loss: 0.4170 - val_acc
```

```
uracy: 0.8445 - val_loss: 0.3899
Epoch 20/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8234 - loss: 0.4173 - val_acc
uracy: 0.8425 - val_loss: 0.3863
```

In [17]:
```python
# Print accuracy per epoch
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

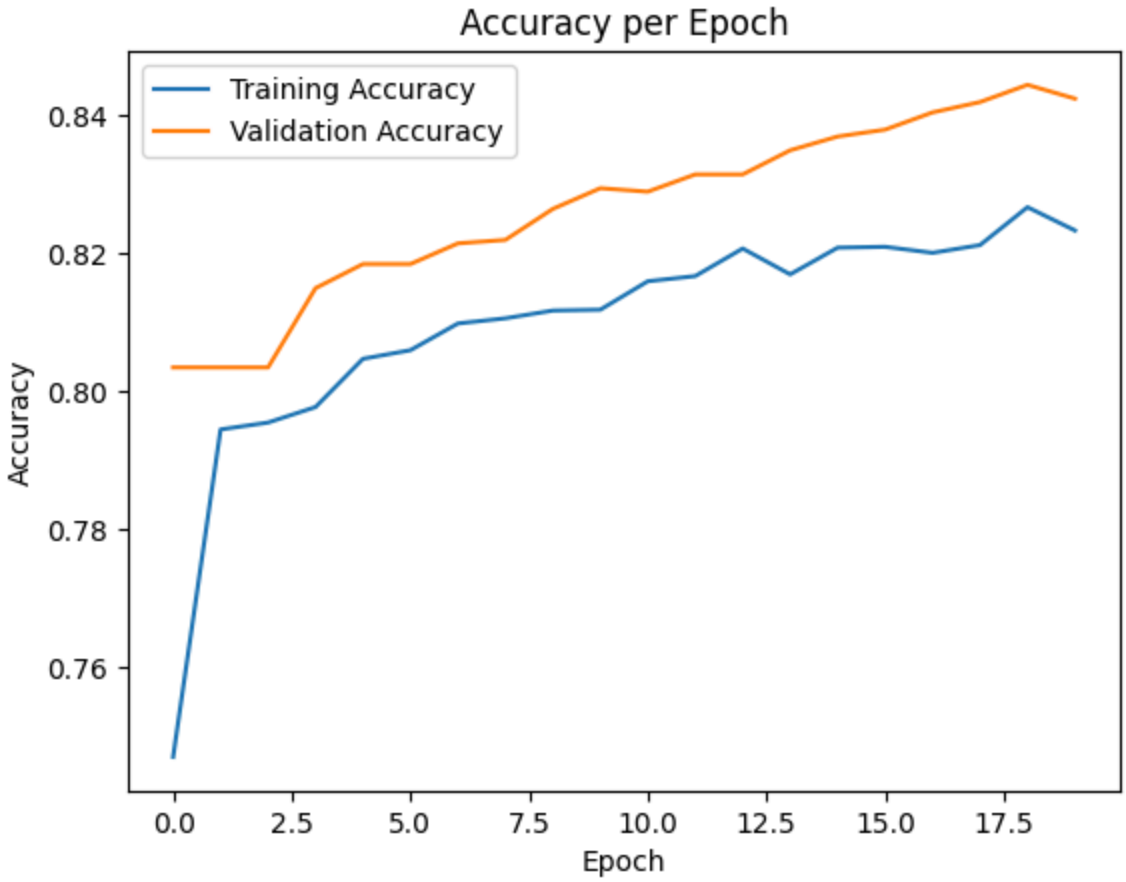
print("Accuracy per Epoch:\n")
for i in range(len(train_acc)):
    print(f"Epoch {i+1}: Training Accuracy = {train_acc[i]*100:.2f}%, Validation Ac
```

```
Accuracy per Epoch:

Epoch 1: Training Accuracy = 74.70%, Validation Accuracy = 80.35%
Epoch 2: Training Accuracy = 79.45%, Validation Accuracy = 80.35%
Epoch 3: Training Accuracy = 79.55%, Validation Accuracy = 80.35%
Epoch 4: Training Accuracy = 79.77%, Validation Accuracy = 81.50%
Epoch 5: Training Accuracy = 80.48%, Validation Accuracy = 81.85%
Epoch 6: Training Accuracy = 80.60%, Validation Accuracy = 81.85%
Epoch 7: Training Accuracy = 80.99%, Validation Accuracy = 82.15%
Epoch 8: Training Accuracy = 81.06%, Validation Accuracy = 82.20%
Epoch 9: Training Accuracy = 81.17%, Validation Accuracy = 82.65%
Epoch 10: Training Accuracy = 81.19%, Validation Accuracy = 82.95%
Epoch 11: Training Accuracy = 81.60%, Validation Accuracy = 82.90%
Epoch 12: Training Accuracy = 81.67%, Validation Accuracy = 83.15%
Epoch 13: Training Accuracy = 82.07%, Validation Accuracy = 83.15%
Epoch 14: Training Accuracy = 81.70%, Validation Accuracy = 83.50%
Epoch 15: Training Accuracy = 82.09%, Validation Accuracy = 83.70%
Epoch 16: Training Accuracy = 82.10%, Validation Accuracy = 83.80%
Epoch 17: Training Accuracy = 82.01%, Validation Accuracy = 84.05%
Epoch 18: Training Accuracy = 82.13%, Validation Accuracy = 84.20%
Epoch 19: Training Accuracy = 82.67%, Validation Accuracy = 84.45%
Epoch 20: Training Accuracy = 82.34%, Validation Accuracy = 84.25%
```

In [18]:
```python
import matplotlib.pyplot as plt
```

In [19]:
```python
# Accuracy per epoch
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy per Epoch')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```



In [21]:
```
y_pred = model.predict(X_test)
```

```
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
```

In [22]:
```
y_pred = (model.predict(X_test) > 0.5).astype("int32").ravel()
```

```
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
```

In [23]:
```
from sklearn.metrics import accuracy_score
```

In [24]:
```
acc = accuracy_score(y_test, y_pred)
print("Final Accuracy: {:.2f}%".format(acc * 100))
```

```
Final Accuracy: 84.25%
```

In [25]:
```
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

```
Confusion Matrix:
 [[1567   40]
 [ 275  118]]
```

In [ ]: